Cover

Spring Boot

Interview Questions & Answers

Core • MVC • Security • JWT

Spring Boot Interview Questions

Question 1: What does @SpringBootApplication do? What's inside it?

> **Interviewer asks:** *"I see you use @SpringBootApplication in your main class. Can you explain what this annotation actually does and what annotations it combines?"*

Your answer:

@SpringBootApplication is a convenience annotation that combines three annotations:

Annotation	What it does
@Configuration	Marks the class as a configuration class (like XML config)
@EnableAutoConfiguration	Tells Spring Boot to automatically configure beans based on dependencies
@ComponentScan	Enables component scanning to find controllers, services, repositories

```
@SpringBootApplication  // Same as @Configuration +
@EnableAutoConfiguration + @ComponentScan
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

What happens when run() is called?

- Creates Spring ApplicationContext
- Scans for components
- Runs auto-configuration
- Starts embedded server (Tomcat by default)
- Registers shutdown hook

Question 2: What's the difference between @Controller, @Service, and @Repository?

> **Interviewer asks:** *"When building a Spring Boot application, you use different annotations for different layers. Can you explain @Controller, @Service, and @Repository? What happens if I accidentally swap them?"*

Your answer:

Annotation	Layer	What it does	Special feature
@Controller	Web/Presentation	Handles HTTP requests, returns views/REST responses	Works with @RequestMapping
@Service	Business/Service	Contains business logic	Clear layer separation
@Repository	Data/Persistence	Interacts with database	Translates SQL exceptions to Spring's DataAccessException

What happens if you swap them?

- Technically, the application will still work because all three are specialized forms of @Component
- But you lose important features:
  - @Repository loses automatic exception translation
  - Code becomes confusing for other developers
  - Breaks layer architecture conventions

```
                @Controller  // Web layer
                public class UserController {
                    @Autowired private UserService userService;
                }

                @Service     // Business layer
                public class UserService {
                    @Autowired private UserRepository userRepository;
                }

                @Repository  // Data layer
                public interface UserRepository extends JpaRepository<User,
Long> {
```

Question 3: Constructor injection vs Setter injection - which one should I use?

> **Interviewer asks:** *"When injecting dependencies, we have constructor injection and setter injection. Which one do you prefer and why?"*

Your answer:

Aspect	Constructor Injection	Setter Injection
When to use	Required/mandatory dependencies	Optional dependencies
Immutability	Can use final fields	Cannot use final
Testing	Easy - pass mocks in constructor	Need to call setters
Circular dependency	Detected at startup	Can be hidden
Spring's recommendation	✅ Recommended	❌ For mandatory deps

```
                // Constructor injection (Recommended for required dependencies)
                @Service
                public class UserService {
                    private final UserRepository userRepository;
                    private final EmailService emailService;

                    // No @Autowired needed (Spring 4.3+)
                    public UserService(UserRepository userRepository,
EmailService emailService) {
                        this.userRepository = userRepository;
                        this.emailService = emailService;
                    }
                }

                // Setter injection (For optional dependencies)
                @Service
                public class NotificationService {
                    private SmsService smsService;

                    @Autowired(required = false)
                    public void setSmsService(SmsService smsService) {
                        this.smsService = smsService;
                    }
```

```
}
```

✅ **Why constructor injection is better:**

- Dependencies are never null
- Supports immutability
- Easier to unit test
- Clearly shows required dependencies
- Detects circular dependencies at startup

Question 4: Explain different bean scopes in Spring. Why is singleton the default?

Interviewer asks: *"Spring beans can have different scopes. Can you explain singleton, prototype, request, and session scopes? And why is singleton the default?"*

Your answer:

Scope	Description	Created per	Use case
singleton	One instance for entire application	Application startup	Stateless beans (services, repositories)
prototype	New instance every time requested	Each injection	Stateful beans
request	One instance per HTTP request	Each HTTP request	Web controllers, request-scoped data
session	One instance per user session	Each user session	Shopping cart, user preferences

Why singleton is default:

- Most Spring beans are stateless (no mutable data)
- Better performance - no object creation overhead
- Less memory usage - single instance shared
- Thread-safe when designed properly
- Matches most common use cases

```
@Component
@Scope("singleton")  // Default - can omit
```

```java
public class SingletonBean {
    // One instance for entire app
}

@Component
@Scope("prototype")
public class PrototypeBean {
    // New instance every time injected
}

@RestController
@Scope("request")
public class RequestScopedController {
    // New instance per HTTP request
}

@RestController
@Scope("session")
public class SessionScopedController {
    // One instance per user session
}
```

Question 5: What is circular dependency and how do you fix it?

> **Interviewer asks:** *"Have you ever encountered circular dependency issues in Spring? What causes it and how do you resolve it?"*

Your answer:

What is circular dependency?
When Bean A depends on Bean B, and Bean B depends on Bean A (directly or indirectly).

```
@Service
public class ServiceA {
    @Autowired
    private ServiceB serviceB;  // A depends on B
}

@Service
public class ServiceB {
    @Autowired
    private ServiceA serviceA;  // B depends on A - CIRCULAR!
}
```

How to fix it:

Solution	How it works	Code example
@Lazy	Create one bean lazily	@Lazy on constructor
Setter injection	Create beans first, set later	Use @Autowired on setter
@PostConstruct	Set after construction	Initialize in @PostConstruct
Redesign code	Split into 3 classes	Best practice

```
// Solution 1: Using @Lazy
@Service
public class ServiceA {
    private ServiceB serviceB;

    public ServiceA(@Lazy ServiceB serviceB) {
```

```
            this.serviceB = serviceB;
        }
    }

    // Solution 2: Setter injection
    @Service
    public class ServiceB {
        private ServiceA serviceA;

        @Autowired
        public void setServiceA(ServiceA serviceA) {
            this.serviceA = serviceA;
        }
    }

    // Solution 3: Redesign (Best approach)
    // Create a third service that handles the shared functionality
    @Service
    public class SharedService {
        // Common logic here
    }

    @Service
    public class ServiceA {
        @Autowired private SharedService shared;
    }

    @Service
    public class ServiceB {
        @Autowired private SharedService shared;
    }
```

> 💡 **Best practice:** Circular dependencies often indicate design problems. Consider refactoring your code.

Question 6: How do you handle different environments (dev, test, prod) in Spring Boot?

> **Interviewer asks:** *"In real projects, we have different configurations for development, testing, and production. How do you manage this in Spring Boot?"*

Your answer:

Using Spring Profiles:
Profiles allow you to define different configurations for different environments.

```
// Development configuration
@Configuration
@Profile("dev")
public class DevConfig {
    @Bean
    public DataSource dataSource() {
        return new H2DataSource();  // H2 in-memory database for
dev
    }
}

// Production configuration
@Configuration
@Profile("prod")
public class ProdConfig {
    @Bean
    public DataSource dataSource() {
        return new MySQLDataSource();  // MySQL for production
    }
}

// Service that only runs in dev
@Service
@Profile("dev")
public class DevOnlyService {
    public void init() {
        System.out.println("Running in development mode");
    }
}
```

application.properties:

```
# Activate profile
spring.profiles.active=dev

# Profile-specific files will be loaded automatically:
# application-dev.properties
# application-test.properties
# application-prod.properties
```

Different ways to activate profiles:

```
# 1. Command line
java -jar app.jar --spring.profiles.active=prod

# 2. Environment variable
export SPRING_PROFILES_ACTIVE=dev

# 3. VM argument
-Dspring.profiles.active=test
```

Profile-specific properties example:

```
# application-dev.properties
server.port=8080
database.url=jdbc:h2:mem:testdb

# application-prod.properties
server.port=80
database.url=jdbc:mysql://production-server:3306/mydb
database.username=produser
database.password=${DB_PASSWORD}  # Use environment variable for
secrets
```

Question 7: What is Spring Boot auto-configuration? How can you disable it?

> **Interviewer asks:** *"Spring Boot auto-configures many things automatically. Can you explain how this works and how to disable specific auto-configurations if needed?"*

Your answer:

What is auto-configuration?
Spring Boot automatically configures beans based on:

- Dependencies in classpath (what JARs you have)
- Property configurations (application.properties)
- Existing beans in the context

Examples of auto-configuration:

- If H2 database is in classpath → automatically configures DataSource and H2 console
- If spring-web is present → configures DispatcherServlet
- If Thymeleaf is in classpath → configures Thymeleaf view resolver

How to disable auto-configuration:

```
                // 1. Exclude specific auto-configuration using annotation
                @SpringBootApplication(exclude =
{DataSourceAutoConfiguration.class})
                public class Application {
                    public static void main(String[] args) {
                        SpringApplication.run(Application.class, args);
                    }
                }

                // 2. In application.properties

spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSource

                // 3. Exclude multiple configurations
                @SpringBootApplication(exclude = {
                    DataSourceAutoConfiguration.class,
                    HibernateJpaAutoConfiguration.class,
                    SecurityAutoConfiguration.class
                })
```

How auto-configuration works internally:

- Spring Boot looks for META-INF/spring.factories files
- These files list all auto-configuration classes

- Each auto-configuration class has @Conditional annotations
- Conditions check for class existence, property values, bean presence
- Only configurations satisfying all conditions are applied

Common @Conditional annotations:

- @ConditionalOnClass - Configures only if class is present
- @ConditionalOnMissingBean - Configures only if bean doesn't exist
- @ConditionalOnProperty - Configures only if property has specific value
- @ConditionalOnWebApplication - Configures only for web apps

Question 8: Explain how Spring MVC handles a request from start to end.

Interviewer asks: *"When a user makes a request to your Spring Boot application, what happens from the moment request hits the server until response is sent back?"*

Your answer:

Front Controller: DispatcherServlet acts as the front controller.
Request flow diagram:

```
1. Client sends HTTP request
     ↓
2. DispatcherServlet receives request (Front Controller)
     ↓
3. DispatcherServlet asks HandlerMapping: "Which controller handles this URL?"
     ↓
4. HandlerMapping returns the appropriate controller
     ↓
5. DispatcherServlet calls the controller method
     ↓
6. Controller processes request, calls services, gets data
     ↓
7. Controller returns ModelAndView (view name + data)
     ↓
8. DispatcherServlet asks ViewResolver: "Find the actual view"
     ↓
9. ViewResolver returns the view (JSP, Thymeleaf, etc.)
     ↓
10. View renders the response (HTML, JSON, XML)
     ↓
11. Response sent back to client
```

```
                @Controller
                public class HomeController {
                    @GetMapping("/hello")
                    public String hello(Model model) {
                        model.addAttribute("message", "Hello World");
                        return "hello";  // View name - resolves to hello.html
or hello.jsp
                    }
                }

                // For REST APIs (returns data directly)
                @RestController
                public class ApiController {
                    @GetMapping("/api/users")
                    public List<User> getUsers() {
```

```
        return userService.findAll();  // Returns JSON directly
    }
}
```

Components explained:

Component	Responsibility
DispatcherServlet	Receives all requests, coordinates everything
HandlerMapping	Maps URLs to controller methods
Controller	Contains business logic, returns ModelAndView
ViewResolver	Finds actual view files (JSP, Thymeleaf)
View	Renders the response

Question 9: When do you use @Qualifier annotation?

> **Interviewer asks:** *"What happens when you have multiple beans of the same type? How does Spring know which one to inject?"*

Your answer:

Problem: When multiple beans of same type exist, Spring doesn't know which one to inject.

```
// Multiple beans of same type
@Configuration
public class DatabaseConfig {

    @Bean
    @Qualifier("mysql")
    public Database mysqlDatabase() {
        return new Database("MySQL",
"jdbc:mysql://localhost:3306/db");
    }

    @Bean
    @Qualifier("mongodb")
    public Database mongoDatabase() {
        return new Database("MongoDB",
"mongodb://localhost:27017/db");
    }

    @Bean
```

```
    @Primary  // Default if no qualifier specified
    public Database defaultDatabase() {
        return new Database("H2", "jdbc:h2:mem:testdb");
    }
}

// Using @Qualifier to specify which bean
@Service
public class UserService {
    private final Database database;

    @Autowired
    public UserService(@Qualifier("mysql") Database database) {
        this.database = database;  // Gets MySQL database
    }
}
```

Question 10: How do you handle async operations in Spring Boot?

> **Interviewer asks:** *"If you have a long-running task like sending emails or processing files, how do you handle it without blocking the main thread?"*

Your answer:

Using @Async annotation:

```
                // Step 1: Enable async support
                @SpringBootApplication
                @EnableAsync  // Add this annotation
                public class Application {
                    public static void main(String[] args) {
                        SpringApplication.run(Application.class, args);
                    }
                }

                // Step 2: Create async service
                @Service
                public class EmailService {

                    @Async  // Runs in separate thread pool
                    public CompletableFuture<String> sendEmail(String to, String
message) {
                        try {
                            // Simulate long-running task
                            Thread.sleep(5000);
                            System.out.println("Email sent to: " + to);
                            return CompletableFuture.completedFuture("Sent to "
+ to);
                        } catch (InterruptedException e) {
                            return CompletableFuture.failedFuture(e);
                        }
                    }

                    @Async
                    public void processFile(String filePath) {
                        // File processing logic
                        System.out.println("Processing file: " + filePath);
                    }
                }

                // Step 3: Use async service
                @RestController
```

```
public class NotificationController {
    @Autowired
    private EmailService emailService;

    @PostMapping("/send-email")
    public String sendEmail(@RequestParam String to) {
        // This returns immediately, doesn't wait for email to
send
        CompletableFuture<String> future =
emailService.sendEmail(to, "Welcome!");
        return "Email sending started. Check console for
result.";
    }
}
```

Question 11: What are Spring Boot Actuators? How do you use them?

> **Interviewer asks:** *"How do you monitor your Spring Boot application in production? What metrics do you track?"*

Your answer:

What are Actuators?
Actuators provide production-ready features to monitor and manage your application. They expose REST endpoints for various metrics.
Setup:

```
<!-- Maven dependency -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Common actuator endpoints:

Endpoint	What it shows
/actuator/health	Application health (UP/DOWN status)
/actuator/info	Custom app info (Version, name)
/actuator/metrics	Memory, CPU, threads
/actuator/loggers	Log levels - configure at runtime

/actuator/env	Environment properties (database URLs)
/actuator/beans	All Spring beans

Configuration:

```
# application.properties

# Enable all endpoints (be careful in production!)
management.endpoints.web.exposure.include=*

# Enable specific endpoints
management.endpoints.web.exposure.include=health,info,metrics

# Show detailed health info
management.endpoint.health.show-details=always

# Custom info
info.app.name=My Application
info.app.version=1.0.0
```

Question 12: How do you implement JWT security in Spring Boot?

> **Interviewer asks:** *"How do you handle authentication and authorization in your Spring Boot applications? Have you used JWT?"*

Your answer:

JWT (JSON Web Token) Workflow:

```
1. User sends login request (username/password)
     ↓
2. Server validates credentials
     ↓
3. Server creates JWT token with user info and expiry
     ↓
4. Server returns token to client
     ↓
5. Client stores token (localStorage/cookie)
     ↓
6. Client sends token in Authorization header for each request
     ↓
7. Server validates token signature
     ↓
8. Server processes request
```

JWT Structure:

```
Header.Payload.Signature

Header: {"alg": "HS256", "typ": "JWT"}
Payload: {"sub": "123", "name": "John", "exp": 1734567890}
Signature: HMACSHA256(
    base64UrlEncode(header) + "." + base64UrlEncode(payload),
    secret
)
```

Dependencies:

```
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
<dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
```

JWT Utility Class:

```
@Component
public class JwtUtil {
    private String secret = "mySecretKey";
    private int expiration = 86400000; // 24 hours

    // Generate token
    public String generateToken(String username) {
        return Jwts.builder()
                .setSubject(username)
                .setIssuedAt(new Date())
                .setExpiration(new
Date(System.currentTimeMillis() + expiration))
                .signWith(SignatureAlgorithm.HS256, secret)
                .compact();
    }

    // Validate token
    public boolean validateToken(String token) {
        try {

Jwts.parser().setSigningKey(secret).parseClaimsJws(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    // Extract username from token
    public String extractUsername(String token) {
        return Jwts.parser()
                .setSigningKey(secret)
                .parseClaimsJws(token)
                .getBody()
                .getSubject();
    }
}
```

Question 13: How do you handle exceptions globally in Spring Boot?

> **Interviewer asks:** *"How do you ensure consistent error responses across your application? How do you handle exceptions globally?"*

Your answer:

Using @ControllerAdvice / @RestControllerAdvice:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handle specific exceptions
    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse
handleNotFound(ResourceNotFoundException ex) {
        return new ErrorResponse(
                "NOT_FOUND",
                ex.getMessage(),
                HttpStatus.NOT_FOUND.value()
        );
    }

    // Handle validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponse
handleValidation(MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult()
                .getFieldErrors()
                .stream()
                .map(error -> error.getField() + ": " +
error.getDefaultMessage())
                .collect(Collectors.toList());

        return new ErrorResponse(
                "VALIDATION_FAILED",
                errors.toString(),
                HttpStatus.BAD_REQUEST.value()
        );
    }

    // Handle all other exceptions (fallback)
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ErrorResponse handleGeneric(Exception ex) {
        return new ErrorResponse(
                "INTERNAL_ERROR",
                "Something went wrong. Please try again later.",
                HttpStatus.INTERNAL_SERVER_ERROR.value()
        );
    }
```

```
}

// Error response class
public class ErrorResponse {
    private String error;
    private String message;
    private int status;
    private long timestamp = System.currentTimeMillis();
    // Constructor, getters, setters
}
```

In Controller:

```
@RestController
public class UserController {

    @GetMapping("/users/{id}")
    public User getUser(@PathVariable Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new
ResourceNotFoundException("User not found with id: " + id));
    }

    @PostMapping("/users")
    public User createUser(@Valid @RequestBody User user) {
        // Validation errors automatically handled
        return userRepository.save(user);
    }
}
```

Benefits of global exception handling:

- Consistent error format across all APIs
- Centralized exception logic
- Cleaner controller code
- Easier to maintain
- Better client experience

Question 14: Explain HTTP status codes. How do you fix 401 error?

> **Interviewer asks:** *"When building REST APIs, we see different HTTP status codes. Can you explain 200, 401, 404, 500? And how do you debug a 401 error?"*

Your answer:

Common HTTP status codes:

Code	Name	Meaning
200	OK	Request succeeded

401	Unauthorized	Not authenticated - missing/invalid token
403	Forbidden	Authenticated but no permission
404	Not Found	Resource doesn't exist
500	Server Error	Server exception - bug in code

How to debug and fix 401 Unauthorized:

```
                // 1. Check if token is present in request
                String authHeader = request.getHeader("Authorization");
                if (authHeader == null) {
                    // No token provided
                    throw new UnauthorizedException("No Authorization header");
                }

                if (!authHeader.startsWith("Bearer ")) {
                    // Wrong format
                    throw new UnauthorizedException("Invalid token format. Use
'Bearer <token>'");
                }

                String token = authHeader.substring(7);

                // 2. Validate token signature
                try {
                    Jwts.parser().setSigningKey(secret).parseClaimsJws(token);
                } catch (SignatureException e) {
                    throw new UnauthorizedException("Invalid token signature");
                }

                // 3. Check if token expired
                Claims claims = Jwts.parser()
                        .setSigningKey(secret)
                        .parseClaimsJws(token)
                        .getBody();

                Date expiration = claims.getExpiration();
                if (expiration.before(new Date())) {
                    throw new UnauthorizedException("Token expired. Please login
again");
                }

                // 4. Verify user exists
                String username = claims.getSubject();
                User user = userRepository.findByUsername(username);
                if (user == null) {
                    throw new UnauthorizedException("User not found");
                }
```

Common 401 scenarios and fixes:

Scenario	Problem	Fix
No token in request	Missing Authorization header	Client must add header
Wrong token format	Should be "Bearer <token>"	Fix token format
Token expired	Token older than expiry	Get new token (login again)
Invalid signature	Token tampered with	Use correct secret

Spring Boot Quick Reference

Key Annotations:

Annotation	Purpose
@SpringBootApplication	@Configuration + @EnableAutoConfiguration + @ComponentScan
@Controller	Web layer - returns views
@RestController	Web layer - returns data (REST APIs)
@Service	Business layer
@Repository	Data layer + exception translation
@Autowired	Dependency injection
@Qualifier	Specify which bean to inject
@Scope	Define bean scope
@Profile	Environment-specific beans
@Async	Async method execution
@Transactional	Database transaction management
@ControllerAdvice	Global exception handling

Bean Scopes:

- singleton - One instance per container (default)
- prototype - New instance each time requested
- request - One instance per HTTP request
- session - One instance per HTTP session
- application - One instance per ServletContext

Common Properties:

```
# Server
server.port=8080
server.servlet.context-path=/api

# Database
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/db
spring.datasource.username=root
spring.datasource.password=secret

# JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Actuator
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always
```

> 📌 **Remember:** Spring Boot's auto-configuration works when dependencies are in classpath. Check spring.factories files for all auto-configurations!

End of Spring Boot Interview Questions

14+ Questions Covered | Complete Reference

Thank You!

Spring Boot Interview Questions

Created by

Java Developer